DecL Cheat Sheet (1/4): agg

An Aggregate compound distribution has the clause structure:

agg <NAME> <LABEL*> <EXPOSURE> <LAYERS*> <SEVERITY> <OCC_RE*> <FREQUENCY> <AGG_RE*> <APPROX*> <ORIENTATION*> <TRAILER*>

Key: <INPUT> user input; lower_case a DecL keyword; CLAUSE a sub-clause; options a|b|c; inf infinity; * optional. Build with build('agg ...').

1. Name, labels & references

agg <NAME> ...
sev <NAME> <SEVERITY> *standalone severity*
... as <LABEL>, ... as "<LABEL>" *display label*

Names match [a-zA-Z][._:~a-zA-Z0-9-]* *and are the identity handle;* as *adds presentation text. A label may also attach to the exposure, a layer, a severity, a reinsurance layer, and an expense group.*

Reference: agg.<NAME>, sev.<NAME>, port.<NAME>, dist.<NAME>. *Clone / transform:* agg <NEW> agg.<OLD>, 2 * agg.X *(homog.)*, 2 @ agg.X *(inhomog.)*, agg.X + 100.

2. Exposure

<EL> loss
<PREMIUM> premium at <LR> lr
<EXPOSURE> exposure at <RATE> rate
<CLAIMS> claims
dfreq <OUTCOMES> <PROBS*> *fixed empirical freq*
<T> years <at <RATE> rate>* *renewal horizon*

Outcomes [1 2 3 4] or [2:10:2]; probs [.5 .25 .25] or omitted = equally likely. years *pairs only with* wait *(box 7); its* rate *is informational and never sizes the count.*

3. Limit / layers (optional)

<LIMIT> xs <ATTACHMENT> <LABEL*>
tower [<BREAKS>]

Occurrence limits on ground-up severity, unlimited reinstatements, conditional on attaching by default.

4. Severity (continuous)

sev <DIST> <MEAN> cv <CV>
sev <DIST> <SHAPE1> <SHAPE2*>
sev sev.<NAME> *built-in*

<SCALE> * SEV + <LOC> *scale & shift*
ssev <NUM> - SEV *reflect (needs* ssev*)*
SEV wts [<W>], SEV wts=<N> *mixture*
SEV splice [<LB> <UB>] *conditional in layer*
SEV ! *unconditional (when* ATTACH>0*)*
ssev ... *spliced / signed severity*

5. Severity (discrete)

dsev <OUTCOMES> <PROBS*>, dsev ... !
sev (d|c)histogram xps [<X>] [<P>]
picks [<X>] [<P>] *match layer picks*

dsev [1:6] is a fair die; xps *gives outcomes and probabilities directly, and attaches only to a histogram.*

6. Frequency

poisson, bernoulli, fixed, geometric, logarithmic, binomial <P>, negbin <VAR_MULT>, neymana <CLAIMS/OCC>, pascal <CV> <CLAIMS/OCC>

mixed <MIX> <SHAPE1> <SHAPE2*>
MIX = gamma|delaporte|ig|sig|sichel|beta

FREQ zt *zero-truncated*; FREQ zm <P0> *zero-modified,* $\Pr(N{=}0){=}p_0$; *trailing* ! *pins the realized mean to the exposure clause instead of the base mean*

7. Renewal waiting time

wait <SEVERITY> <LABEL*>
wait <Y> xs <A> <SEVERITY> <LABEL*>
dwait <OUTCOMES> <PROBS*>, dwait ... !

The Sparre-Andersen alternative to a frequency clause: the count comes from the waiting-time law over the years *horizon.* wait *takes the full severity mini-language on the time axis. The layered form is conditional by default;* ! *collapses* $\Pr(W \le a)$ *into a zero-wait cluster atom.*

agg R 10 years sev lognorm 50 cv 2 wait
expon 1

8. Reinsurance

occurrence (net of|ceded to) LAYERS
aggregate (net of|ceded to) LAYERS

LAYER = <LIMIT> xs <ATTACH>
| <PART> po <LIMIT> xs <ATTACH>
LAYERS = LAYER and LAYER ... *or* tower [...]

po = part of, the one way to write a partial placement: a percentage such as 50% is the share, a plain number is the amount placed, and the share is amount / limit. Occurrence applies per claim, aggregate to the annual total. Layers may also carry economics and a label; *see page 2.*

9. Approximate & orientation

approximate exact *(default)*, approximate sgamma *shifted gamma*, approximate slognorm *shifted lognormal*

Alias approx*. Replaces the freq × sev convolution with one severity matched to the first three aggregate moments.*

loss *(default),* payoff*. The orientation suffix, last before the trailer:* payoff *marks an asset-return primitive (more is better) and flips pricing to the dual distortion.*

tweedie <MEAN> <P> <DISP> *a declaration on its own*

10. Trailer, vectors, math

Trailer note{free text}, tags{cat, property}, hints{bs=100; log2=17}, doc{{{markdown}}}. *Order-free, at most one of each; only* doc *may carry blank lines and fenced code.*

Vectorize [1 2 3] claims; [250 250 500] xs [0 250 500]; lognorm [1 3] cv [.7 1.5] wts [.6 .4]. *Vectors broadcast; layers / exposure zip.*

Math 1/2, 3**4, 3^4, exp(2), 12.4%, 10_000. *Minus binds to the number, so* $-4^2 = 16$; * *is severity scaling, not general arithmetic.* **Comments #** *to end of line; statements separate on a blank line or* ;.

aggregate v.1.2.0a14
ease | confidence | precision
© Stephen J. Mildenhall
2026-10-13 16:42

DecL Cheat Sheet (2/4): reinsurance economics, `pnl` and `xpnl`

Page 1 declared the reinsurance *loss* structure. Everything here is *money*: ceded premium, commission, reinstatements, variable rating, and the profit-and-loss statement that books them.
A priced layer is `LAYER <PREMIUM*> <CEDE*> <REINSTATEMENTS*> <VARIABLE*> <LABEL*>`, in that order.

1. Ceded premium & commission

```
LAYER deposit <AMOUNT>
LAYER rol <RATE> rate on line × share × limit
LAYER rate <FRACTION> of gross premium

...   cede <FRACTION> ceding commission
```

The three premium forms are mutually exclusive. `cede` *is valid only with a premium; the premium is entered* gross *of commission.*

```
occurrence net of 250 xs 250 deposit 60 cede 20%
```

2. Reinstatements

```
reinstatements [0 .5 1 1] explicit multipliers
reinstatements 1 free and 2 at 100%
no reinstatements one annual limit
```

Multipliers α_j are relative to the base rate on line r, so a base premium clause is required. `free` $=$ `at 0%`. *Counts may be digits or the words* `one`…`five`. *m reinstatements* $= m + 1$ *annual limits, so* `no reinstatements` *is one limit — distinct from* omitting *the clause, which keeps free and unlimited.*

One occurrence layer, one clause. This is the stochastic-ceded property-cat feature.

3. Variable rating

```
swing basic <B> lcm <M> <min <LO>* <max <HI>*
slide <C1> at <LR1> and <C2> at <LR2> ...
pc <SHARE> after <ALLOWANCE>
corridor <SHARE> po <WIDTH> xs <ATTACH>
```

`swing` replaces *the deposit / rol / rate premium with the collared affine* clip$(b + m \cdot L_{ceded}, lo, hi)$; `slide` replaces `cede` *with a commission sliding on the ceded loss ratio;* `pc` *is a profit commission on the ceded premium;* `corridor` *is a loss-ratio band the cedant retains.*

At most one *variable feature per program (no stacking), and* aggregate reinsurance only *in this release.*

4. `pnl` / `xpnl` declaration

```
pnl <NAME> <LABEL*> <PREMIUM> less <ENGINE>
<less <EXPENSES>* <TRAILER*>
xpnl <NAME> ... same syntax
```

Profit and loss = premium minus what comes out of the sausage maker. `less` *is a dedicated keyword, never −, so the split cannot collide with severity arithmetic (*`ssev 20 - lognorm`*); a* second `less` *is the hard anchor introducing expenses.*

```
pnl P 1000 premium less agg E 700 loss sev
lognorm 50 cv 2 poisson
```

5. The premium head

```
<AMOUNT> premium <LABEL*>
inherit premium <LABEL*>
derive premium <LABEL*>
retro <COLLAR> premium <LABEL*>
```

The premium is the consideration *(what you book); the engine's own* `premium at lr` *is a separate* sizing *input.* `inherit` *copies the engine's technical premium (a build error if it has none).* `derive` *grosses it up for the expense clause,* $P = (T + fixed)/(1 - premium\ ratios)$, *so premium net of expenses returns exactly T; fixed and premium bases only.* `retro` *is the account-level retrospective-rating clause, taking the same* `basic ... lcm ... min ... max` *collar as* `swing`.

```
pnl P inherit premium as "gross written"
less agg.Book
```

6. The engine source

```
agg <NAME> <AGG-BODY> inline
agg.<NAME> stored aggregate
port.<NAME> stored portfolio (net-net total)
```

A P&L wraps a complete *stochastic engine, never an anonymous half-body: the inline form reuses the identical* `agg` *body from page 1, minus the trailer, which the wrapping* `pnl` *owns. So every clause on page 1 is available inside a* `pnl`.

7. Expenses

```
less <TERM> and <TERM> ...   <LABEL*> one group
less <GROUP> <GROUP> ... several groups

TERM = <F> premium expense
  | <F> loss expense
  | <AMOUNT> fixed expense
```

`expense` *and* `expenses` *are both accepted. Terms joined by* `and` combine *into one reported obligation leg; juxtaposed groups stay* separate *items. Each group may carry its own* `as` *label; the default name is the basis.*

```
less 25% premium expense as acquisition 1000
fixed expense as overhead
```

8. `pnl` vs. `xpnl`

`pnl` *the* consolidated *single-group net view: one* `Consideration` / `Obligation` / `Margin` *card.*

`xpnl` *the* exploded *sibling, same syntax: a multi-group walk with one group per step, so you see gross, then each cession, then expenses, as separate blocks that foot to the same margin.*

Both return a PnL. Reach for `xpnl` *to see* where *the margin went, for* `pnl` *to see* what it is. *Both may be portfolio units.*

9. Worked example

```
xpnl Book 10000 premium
  less agg Book_e 10000 premium at 75% lr
    10000 xs 0
    sev 10.808 * lognorm 1.75
    mixed gamma 0.25
    aggregate net of 5000 xs 10000
      deposit 1500 pc 50% after 10%
  less 20% premium expense
  note{profit commission on the agg cover}
```

Premium head, engine with an aggregate cession carrying a deposit and a profit commission, then expenses.

`aggregate v.1.2.0a14`
ease | confidence | precision
© Stephen J. Mildenhall
2026-10-13 16:42

DecL Cheat Sheet (3/4): `port` and the bivariate family

These declarations build collections and couplings of aggregates. Each component is an ordinary `agg` (or `pnl`) declaration, so everything on pages 1 and 2 is available inside them.

1. Portfolio (`port`)

```
port <NAME> <LABEL*> <TRAILER*>
  agg <UNIT1> ...
  agg <UNIT2> ...
  ...
```

A `Portfolio` is a list of independent *units sharing a discretization grid; the total is their convolution. Units may be `agg` or `pnl` lines, or references `agg.<NAME>`. Per-unit reinsurance is set on each unit line.*

The portfolio trailer sits before *the unit list, so a trailing trailer always binds to the last unit.*

2. `port` example

```
port MyBook note{two lines}
  agg A 100 claims sev lognorm 50 cv 2
poisson
  agg B 200 claims sev gamma 30 cv 1 poisson
```

Access units by name (`p.A`); `unit_names_ex` adds `total`.

3. Bivariate (`bv`)

```
bivariate <NAME> <SHARED-EXPOSURE>
  agg <A> ...
  agg <B> ...
  copula <KIND> <P*>
  <FREQUENCY*>
```

Abbreviation `bv`. Couples exactly two *component severities by a copula and accumulates them with a* shared *outer frequency via a 2D FFT. The shared exposure may be `<N> claims` or `dfreq [...]`; the trailing frequency is optional (default poisson). Per-event triggers come from the inner `dfreq [0 1] [p0 p1]` Bernoulli forms.*

4. Copula clause

```
copula <KIND> <P> one-parameter
copula <KIND> parameter-free
omitted = copula independent

normal <rho> ρ ∈ (−1, 1)
gumbel <tau> τ ∈ [0, 1) upper tail dependence
clayton <tau> τ ∈ [0, 1) lower tail dependence
fgm <rho_s> ρ_s ∈ [−1/3, 1/3]
independent

bv M 100 claims agg A ...  agg B ...  copula
gumbel 0.4
```

5. Discrete bivariate (`dbvsev`)

```
dbvsev [<XS>] [<YS>] [[<ROW>] [<ROW>] ...]
dbvsev [<XS>] [<YS>] uniform on the lattice
dbvsev [[<X> <Y> <P>] ...] sparse triples
```

The 2-D analogue of `dsev`: the joint per-claim probability matrix given directly, `matrix[i][j]` = $\Pr(X{=}x_i, Y{=}y_j)$. Replaces the two `agg` bodies and *the copula clause. The three surface forms are auto-detected.*

```
bv D 10 claims dbvsev [1 2] [3 4] [[.25 .25]
[.25 .25]]
```

6. View pairs

```
netceded <AGG>
grossceded <AGG>
grossnet <AGG>
```

One ordinary `agg` carrying occurrence reinsurance; the keyword names the pair x-then-y *out of {gross, ceded, net} (gross leads when present). The two coupled marginals are positions of the* same *loss, so their dependence is exact, comonotone within the layer. Lets you see the pair* jointly, *not just their marginals.*

```
netceded agg G 100 claims sev lognorm 50 cv
2 occurrence net of 500 xs 500 poisson
```

7. Clash

```
clash <NAME> <NA> <NB> <NC> claims
  <LAYERS*> <SEVERITY> component A
  <LAYERS*> <SEVERITY> component B
  <FREQUENCY*>
```

Dependence from interpretable claim counts rather than a copula parameter: n_a claims hit A, n_b hit B, and n_c hit both *(the clash). The solver derives the shared event count n_0 and the two per-event Bernoulli triggers p_a, p_b.*

```
clash C 10 8 2 claims 100 xs 0 sev lognorm
50 cv 2 100 xs 0 sev gamma 40 cv 1 poisson
```

`aggregate v.1.2.0a14`
ease | confidence | precision
© Stephen J. Mildenhall
2026-10-13 16:42



DecL Cheat Sheet (4/4): `distortion`

A distortion g defines a coherent risk measure / pricing functional. Declare one by kind and its natural parameters; the kind → parameter mapping lives on each `Distortion` subclass (`decl_params`), so the grammar holds no per-kind knowledge.

1. Declaration

```
distortion <NAME> <KIND> <PARAMS> <TRAILER*>
```

Alias `dist`. `<PARAMS>` is a flat list of the kind's natural parameters, in the order of the table. Reference a stored distortion as `dist.<NAME>`. A `dist` line carries the same trailer as every other statement, so it can be described, tagged, and documented.

```
distortion D1 ph 0.9
dist D2 wang 0.25 note{market standard}
dist D3 power 0 100 2
```

2. Kinds & parameters

One shape parameter:

`ph <a>` *proportional hazard*
`wang <lam>` *Wang / normal shift*
`dual <b>` *dual moment*
`tvar <p>` *tail VaR at p*
`ccoc <r>` *constant cost of capital*
`cll <b>` *capped log-linear*
`clin <slope>` *capped linear*
`lep <r>` *leverage-equivalent pricing*
`ly <r>` *linear yield*

Several parameters:

`bitvar <p0> <p1> <w1>` *weight* w_1 *on* p_1
`beta <a> <b>` a *in* $(0, 1]$
`power <x0> <x1> <alpha>`

3. Combinators

```
distortion <NAME> minimum dist.<A> dist.<B>
...
distortion <NAME> mixture dist.<A> dist.<B>
...  wts [<W>]
```

`minimum` = pointwise min of the referenced distortions (min of concaves is concave, so the result is a distortion); `mixture` = weighted average, weights via `wts`. Both take distortion references*, not numbers.*

4. Not declarable

`wtdtvar` *(weighted TVaR) has* no *DecL number-list form*: it takes parameter vectors *of knots and weights. Build it in Python, or reach it through the combinators.*

```
Distortion.random_distortion(n_knots=5)
Distortion.mixture([d1, d2], [.6, .4])
```

List the registered kinds with
`Distortion.available_distortions()`.

5. Usage

Apply a distortion to price an aggregate or portfolio:

```
a = build('agg A 100 claims sev lognorm 50
cv 2 poisson')
d = build('distortion D ph 0.9')
a.price(0.99, d)
p.price(0.99, d) total and allocation by unit
```

Calibrate to a target instead of guessing a shape:
`p.calibrate_distortions(coc=0.1, p=0.99)`.

A distortion satisfies $g(0) = 0$, $g(1) = 1$, *increasing and concave; the dual is* $g^*(x) = 1 - g(1 - x)$*; the distorted price is* $\int_0^\infty g(S(x))\,dx$.

6. And back again

The reverse question — which *distortions are consistent with an observed price — is not a declaration but an analysis. See the `*Bounds` card: its answers come back as `bitvar` distortions, which* are *declarable.*

```
ab = p.allocation_bounds(p=0.99)
ab.bitvars(1200)
dist D bitvar 0.5 0.99 0.3
```

`aggregate v.1.2.0a14`
ease | confidence | precision
© Stephen J. Mildenhall
2026-10-13 16:42

`Underwriter` Class Cheat Sheet

m `Underwriter(*, name='Rory', databases=None, update=cfg, log2=cfg, debug=False)`[1]

The `Underwriter` manages the creation of `Aggregate`, `Severity`, `Portfolio`, `BivariateAggregate`, `PnL`, and `Distortion` objects, maintains a base of named DecL declarations (`Recipe` records keyed by `(kind, name)`), and provides the interface to the Lark-based DecL parser. The convenience method m `build`[2] is the primary entry point; `uw(program)` delegates to it. The following tables show all m `methods`, and fields or properties (used interchangeably). Comments elucidate the meaning of more obscure entries.

1. Specification & creation

`name` *identifying name for this Underwriter instance*
`databases` *the load* request*: name / path / glob, or `'default'`, `'user'`, `'all'`; a bare `Underwriter()` loads nothing*
`update` *if true, constructed objects are updated (probabilities computed); defaults to config*
`log2` *default* $\log_2$ *of the number of buckets used to discretize; defaults to config*
`debug` *if true, print parser debug messages*

2. Update

m `build`[2] (primary entry point: parse a single-output DecL program and return the constructed object), m `build_many` (parse a multi-output program, construct each, smart-update; returns `list[Recipe]`), `recipes` (the recipe base as a `(kind, name)`-indexed `DataFrame`, lazy-loaded), m `recipe` (the one canonical lookup: one `Recipe` by name; `uw[name]` and `uw[kind, name]` delegate to it)[3], m `load` (resolve a request and read matching `.agg` files into the recipe base), m `add_recipe` (add or overwrite one parsed declaration), m `to_agg` (export a selection of recipes to a `.agg` file; re-loads cleanly), m `reload` (reset to as-created state and re-read the configured request), `lexer`, `parser` (the DecL lexer / parser objects)

3. Moments

None

4. Statistical functions

None

5. Validation

m `interpret_file` (parse every DecL program in a `.agg` / `.csv` file and return per-line error info; collects *all* errors rather than aborting — validate a file before installing it), `test_suite_file` (path to the bundled test suite `.agg`, or `None`)[4]

6. Output dataframes

`recipes` *(the only frame: one row per stored declaration; see Update)*

7. Reinsurance

None

8. Visualization

None (though m `discover(plot=True)` plots every match)

9. Risk and pricing

None

10. Approximations

None

11. Meta

m `discover` (search the recipe base by `regex`, `tags`, and optional `kind`; optionally describe, plot, or return the objects — the directory view), m `help` (help on methods and properties matching `regex`), m `available_databases` (list `.agg` files on the search path: cwd / user / default), m `resolve_databases` (dry run: preview the files a request *would* load), m `show_settings` (print every resolved setting and where its value came from), m `reload_settings` (re-read the config file / environment), m `write_default_config` (write the annotated config template to `user_dir`), `default_dir` (bundled `.agg` databases), `user_dir` (`~/.aggregate`), `version` (`aggregate` version string)

Notes:

[1]: Arguments are **keyword-only** (leading `*`): use `databases='test_suite'`, not `Underwriter('test_suite')`. `update` and `log2` default to the `[build]` section of `config.toml`.

[2]: Also exported as `from aggregate import build`. The module-level `build` pre-loads `build.databases` (`test_suite`) whereas a bare `Underwriter()` starts empty. `build` returns one object, `build_many` a list.

[3]: A `Recipe` is one declaration record: `kind`, `name`, `spec`, `program`, `source`, the trailer (`note`, `tags`, `hints`, `doc`), the canonical `decl` re-rendering, and the cookbook sections (`problem`, `solution`, `discussion`, `check`). Lookup is pure: `Recipe.object` is `None` until `build` runs.

[4]: `aggregate/agg/test_suite.agg` (categories A–O) is exercised by `uv run pytest`, not by an `Underwriter` method.

`kind` ∈ {`sev`, `agg`, `port`, `distortion`, `bvagg`, `pnl`, `xpnl`, `expr`}; database search path cwd → `user_dir` → `default_dir`.

`aggregate v.1.0.0a171`
ease | confidence | precision
© Stephen J. Mildenhall
2026-07-29 11:33

`Severity` Class Cheat Sheet

m `Severity(sev_name, exp_attachment=None, exp_limit=inf, sev_mean=0, sev_cv=0, sev_a=nan, sev_b=0, sev_loc=0, sev_scale=0, sev_xs=None, sev_ps=None, sev_wt=1, sev_lb=0, sev_ub=inf, sev_conditional=True, sev_signed=False, sev_reflect=False, name='', note='', hints='', tags=(), doc='', label=None, label_map=None)`

A `Severity` wraps a frozen `scipy.stats` rv (in `fz`) for a single ground-up loss, optionally cut to a layer by the attachment / limit clause. Mixtures and the `wt` weight are handled by `Aggregate`. The following tables show all m `methods`, and fields or properties (used interchangeably). Comments elucidate the meaning of more obscure entries.

1. Specification & creation

`name` *object identity*; `label` *display*
`sev_name` *scipy dist name, or* `(c|d)histogram`
`sev_kind`, `long_name` *resolved labels*
`sev_mean`, `sev_cv` *ground-up mean / CV, or*
`sev_a`, `sev_b` *shape parameters 1, 2 (as declared)*
`a`, `b` *resolved shapes handed to scipy*
`sev_loc`, `sev_scale` *shift and scale*
`sev_xs`, `sev_ps` *(d)histogram outcomes / probs*
`sev_wt` *mixture weight (used by* `Aggregate`*)*
`sev_reflect` *reflect about 0*
`program`, `pprogram` *canonical DecL*

2. Update

`exp_attachment`, `exp_limit` *layer inputs*
`attachment`, `limit`, `detachment` *resolved layer*
`sev_lb`, `sev_ub` *conditional support bounds*
`conditional` *losses conditional on attaching the layer (default) or unconditional*
`signed` *negative outcomes allowed*
`pattach`, `pdetach` Pr*(attach),* Pr*(detach)*
`moment_pattach` *attach probability used in the moment integrals*

3. Moments

`sev1`, `sev2`, `sev3` (first three raw moments of the layered severity), `actual_m`, `actual_cv`, `actual_sd`, `actual_var`, `actual_skew` (analytic, post-layer, post-splice), m `moms` (m, cv, skew), m `mean`, m `median`, m `std`, m `var`, m `stats`, m `moment`, m `generic_moment`, m `moment_type`, m `support`

4. Statistical functions

m `cdf`, m `sf`, m `pdf`, m `logcdf`, m `logsf`, m `logpdf`, m `ppf`, m `isf`, m `rvs`, m `interval`, m `expect`, m `entropy`, m `vecentropy`, m `nnlf`

5. Validation

`bounded` (post-layer, post-splice support is bounded), `tail_class`, `tail_description`, `tail_explanation`
No moment / FFT validation: a `Severity` *is exact, nothing is discretized until* `Aggregate` *builds the grid.*

6. Output dataframes

`info` (fixed-layout text summary)
No dataframes: there is no grid to tabulate. Discretized severity output lives on `Aggregate` *(*`sev_density_df`*).*

7. Reinsurance

None (layer cut only; see Update)

8. Visualization

m `plot`

9. Risk and pricing

None

10. Approximations

m `fit`, m `fit_loc_scale`, m `freeze` (inherited from the scipy rv)

11. Meta

`fz` (the frozen scipy rv), `shapes`, `numargs` (scipy shape info), m `help`, m `format_program`, `pprogram_html`, `note`, `tags`, `doc`, `hints` (the DecL trailer), `label_map`, `labels`, `renamer`, `use_labels`, `random_state`, `xtol`, `badvalue`

Notes:

A `Severity` models one ground-up severity curve; frequency, mixing, and reinsurance live on `Aggregate`.

`sev_mean+sev_cv` or `sev_a(+sev_b)` parameterise the shape; `sev_loc`/`sev_scale` then shift/scale.

The layered severity is $X(a,d) = \min(d, (X-a)^+)$ with attachment a and limit d; `sev1`, `sev2`, `sev3` are its first three raw moments and `actual_*` the derived summary statistics.

Abbreviations: m=mean, cv=coefficient of variation, sd=standard deviation, var=variance, skew(ness).

`aggregate v.1.0.0a171`
ease | confidence | precision
© Stephen J. Mildenhall
2026-07-29 11:33

`Aggregate` Class Cheat Sheet

m `Aggregate(name, exp_el, exp_premium, exp_lr, exp_en, exp_attachment, exp_limit, exp_years, exp_rate, sev_name, sev_a, sev_b, sev_mean, sev_cv, sev_loc, sev_scale, sev_xs, sev_ps, sev_wt, sev_lb, sev_ub, sev_conditional, sev_signed, sev_reflect, sev_pick_attachments, sev_pick_losses, occ_reins, occ_kind, freq_name, freq_a, freq_b, freq_zm, freq_p0, freq_pin_mean, wait_*[0], agg_reins, agg_kind, reins_bucket, dsev_bucket, value_type, approximate, label, label_map, note, hints, tags, doc)`

The constructor follows the DecL clauses (exposure, severity, occurrence reinsurance, frequency, aggregate reinsurance, approximate). Usually built via `build('agg ...')`. The following tables show all m `methods`, and fields or properties (used interchangeably). Comments elucidate the meaning of more obscure entries.

1. Specification & creation

`name` (identity), `label` (display), `limit`, `attachment`, `frequency`, `sevs` (list of `Severity`), `en` (expected claims), `n` (frequency), `base_mean` (un-modified E[N], before `zt`/`zm`), `value_type` (loss / payoff), `exp_premium`, `exp_lr`, `exp_rate`, `exp_years`, `sev_pick_attachments`, `sev_pick_losses`, `program`, `pprogram` (canonical DecL), `spec`, `spec_ex`

Derived objects: m `make_pnl` (wrap as the risky leg of a PnL), m `create_frequency` (claim count as its own `Aggregate`)

2. Update

`log2`, `bs` (bucket size), `sev_calc` (round / forward / backward), `discretization_calc` (distribution / survival / both), `normalize`, `padding`, `dsev_bucket`, `reins_bucket` (atom placement schemes), `x_min`, `x_max`, `i0` (window), m `update`, m `update_work`, m `discretize`, m `picks` (hit layer loss picks), m `rescale` (homog. sev / inhomog. freq), m `snap`, m `unwrap` (undo FFT wrapping)

3. Moments

`actual` = analytic, `est` = estimated from the FFT grid

`actual_m`, `actual_cv`, `actual_sd`, `actual_var`, `actual_skew`, `est_m`, `est_cv`, `est_sd`, `est_var`, `est_skew`, `sev_m`, `sev_cv`, `sev_sd`, `sev_var`, `sev_skew`, `est_sev_m`, `est_sev_cv`, `est_sev_sd`, `est_sev_var`, `est_sev_skew`, m `freq_pmf`

4. Statistical functions

m `cdf`, m `sf`, m `pdf`, m `pmf`, m `q` (lower quantile = VaR), m `tvar`, `sev` (exact severity cdf / sf / pdf), m `q_sev`, m `tvar_sev`, m `var_dict`[1], `prob_eq_0` (Pr(X=0)), m `sample`

5. Validation

`valid`, `validation_eps`, `validation_df` (moment vs. estimate, the QA view), `validation_explanation`, m `aggregate_error_analysis` (agg error over `bs`), m `severity_error_analysis` (trunc. / discr. by component), `bs_window_df`, `bs_description`, `bs_explanation`, `tail_class` ((freq, sev, agg) triple), `tail_behavior_df`, `tail_description`, `tail_explanation`, `bounded`

6. Output dataframes

`density_df`[1] (main output), `density` (live rows only), `summary_df` (at a glance: moments + percentiles), `stats_df`, `info`, `tail_df`, m `tail_periods_df` (return periods), m `focus` (central window holding $1-p$ of the mass), `agg_density`, `sev_density`, `sev_density_df`, `ftagg_density` (FT), `xs`, `xs_sev`, `figure` *see also Reinsurance*

7. Reinsurance

`occ_kind`, `occ_reins`, `agg_kind`, `agg_reins`, `agg_density_gross/ceded/net`, `sev_density_gross/ceded/net`, `occ_ceder`, `occ_netter`, `agg_ceder`, `agg_netter`, m `apply_occ_reins`, m `apply_agg_reins`, `reins_kinds`, `reins_description`, `reins_summary_df` (per stage, the daily driver), `reins_density_df`, `reins_stats_df`, m `occ_bivariate` (joint law of two of gcn; see the `BivariateAggregate` card)

8. Visualization

m `plot`, m `reins_occ_plot`, `figure`

9. Risk and pricing

m `price`, m `price_ccoc`, m `price_pentagon`, m `price_pentagon_ex` (complete the pricing octet), m `prob_loss_assets` (consistent (p, L, a) triple), m `apply_distortion`, m `calibrate_distortions` (standard set to a CoC target), m `pollaczeck_khinchine` (probability of eventual ruin, Poisson), m `wiener_hopf` (ruin for a renewal / Sparre-Andersen model)

10. Approximations

`approximation` (the stored kind: exact / sgamma / slognorm), m `approximate` (fit by moment matching)[2], m `entropy_fit`, `fzapprox`

11. Meta

m `help`, m `json` (persist), m `format_program`, `pprogram_html`, `note`, `tags`, `doc`, `hints` (the DecL trailer), `label_map`, `labels`, `renamer`, `use_labels` (the label surface)

Notes:

[0]: `wait_*` is the renewal (Sparre-Andersen) waiting-time family: `wait_name`, `_a`, `_b`, `_mean`, `_cv`, `_loc`, `_scale`, `_xs`, `_ps`, `_wt`, `_lb`, `_ub`, `_conditional`, `_attachment`, `_limit`. It pairs with `exp_years`, never with a frequency clause.

[1]: matches `Portfolio`. [2]: `approximate` is the *action*; `approximation` is the *stored kind* set by the DecL clause.

Abbreviations: gcn = gross (subject), ceded, net; actual = analytic, est = FFT estimate.

`aggregate v.1.0.0a171`
ease | confidence | precision
© Stephen J. Mildenhall
2026-07-29 11:33

aggregate

`BivariateAggregate` Class Cheat Sheet

m `BivariateAggregate(name, units=None, copula=None, note='', hints='', tags={}, doc='', mode='copula', nc_agg, nc_kwargs, nc_views, clash, dbv_xs, dbv_ys, dbv_S, exp_en, exp_el, exp_premium, exp_lr, freq_name='poisson', freq_a, freq_b, freq_zm, freq_p0)`

A `BivariateAggregate` is the joint law (A_0, A_1) of two coupled aggregates. A **shared** outer frequency drives both; within each event the two per-claim severities are coupled, and the joint is accumulated by a 2D FFT. Either marginal reproduces the standalone `Aggregate`. Usually built via `build('bivariate ...')` (alias `bv`); see the DecL card, page 3. The following tables show all m `methods`, and fields or properties (used interchangeably). Comments elucidate the meaning of more obscure entries.

1. Specification & creation

`name`, `units` (the two component `Aggregates`), `unit_names`, `copula` (the fitted `Copula`), `frequency`, `freq_name` (the *shared* outer count), `en` (expected event count), `clash` (the solved $(n_a, n_b, n_c, n_0, p_a, p_b)$ when declared with `clash`), `program`, `pprogram`, `mode`[1]

Four declaration modes: `copula` (two `agg` bodies + a `copula` clause), `discrete` (`dbvsev`, the joint per-claim matrix given directly), view-pair (`netceded` / `grossceded` / `grossnet` over one reinsured `agg`), `clash` (counts n_a, n_b, n_c solved into a shared count and two Bernoulli triggers).

2. Update

m `update(log2, bs, padding, store_dir, row_chunk, col_chunk, keep_transform)`, m `update_work`, `bs` (per-axis bucket sizes, a *list*), `axis_xs` (per-axis output grids), `padding`

Scalar `log2` is the total *2-D cell budget, split between the axes by measured support; a 2-tuple pins the per-axis `log2`. `store_dir` switches to the* massive *disk-backed path (extra `aggregate[massive]`), where `padding` defaults to 0.*

3. Moments

m `moments` (mixed raw moments $\mathsf{E}[A_0^i A_1^j]$), `corr` (Pearson correlation of the two aggregates), `dependency_df` (`cov` / `corr` / `tau` at both levels: per-claim `Sev` and `Agg`), `stats_df` (per-component marginal moments, theory vs. empirical)

4. Statistical functions

Via the `bivariate` *view*[2]: m `pushforward` (law of $z = f(A_0, A_1)$ on a fresh 1-D grid — the workhorse: *any* scalar function of the pair), m `transformed_moments` (exact raw and central moments of $f(A_0, A_1)$, no regridding), m `moments`, m `corr`, m `marginals`

Univariate risk measures come from the pushforward or from either marginal `Aggregate`; a 2-D law has no single quantile.

5. Validation

`summary_df` (two-unit `Portfolio`-shape summary: theory vs. realised), `validation_explanation`, `deficit` (mass lost off the grid corner)[3], `bs_window_df`, `bs_description`, `bs_explanation` (per axis), `tail_df`, `tail_description`, `tail_explanation`

6. Output dataframes

`density` (the joint pmf, shape (n_0, n_1)), `density_df` (the same as a `DataFrame`), `marginals` (the two 1-D densities), `bivariate` (the `BivariateDistribution` view)[2], `info`, `summary_df`, `stats_df`, `dependency_df`, `axis_xs`, `figure`

7. Reinsurance

The whole point of the view-pair modes. `netceded` `<AGG>` (and `grossceded`, `grossnet`) build the joint occurrence law of two of {gross, ceded, net} for one reinsured aggregate — exact dependence, comonotone within the layer. The object-API route is m `Aggregate.occ_bivariate(views)`.

Per-component reinsurance is declared on each inner `agg` line as usual.

8. Visualization

m `plot` (two-panel contour: per-claim severity, then aggregate), m `bivariate.contour` (one filled contour panel), `figure`

9. Risk and pricing

None directly. Push the joint through the payoff you want to price — `bivariate.pushforward(f)` returns an ordinary 1-D distribution — then price that, or wrap it in a `PnL` (a `BivariateDistribution` is an accepted P&L `source`).

10. Approximations

None. The 2-D FFT is exact on the grid; the only approximation levers are the axis budgets (`log2`, `bs`) and `padding`, in Update.

11. Meta

m `help`, m `format_program`, `pprogram_html`, `note`, `tags`, `doc`, `hints` (the DecL trailer)

Notes:

[1]: `mode` is `'copula'` or `'discrete'`; the view-pair and `clash` statements set the remaining constructor arguments (`nc_agg` / `nc_views`, `clash`).

[2]: `bivariate` is a `BivariateDistribution`: `density`, `axis0` / `axis1` (positionally `ceded` / `net`), `axis_names`, `bs_ceded` / `bs_net`, `meta`, and m `marginals`, m `moments`, m `corr`, m `contour`, m `pushforward`, m `transformed_moments`. Address axes *by role* (`axis0` / `axis1` / `axis_names`), never by the positional names.

[3]: With `padding=0` the FFT wraps rather than losing mass, so `deficit` can read clean while the answer is wrong: compare means, not `deficit`.

`aggregate v.1.0.0a171`
ease | confidence | precision
© Stephen J. Mildenhall
2026-07-29 11:36

Portfolio Class Cheat Sheet

m Portfolio(name, spec_list, uw=None, label=None, label_map=None, note='', tags=(), doc='')

A Portfolio is a collection of Aggregate units sharing a discretization grid; it adds diversification, capital allocation, and distortion pricing. spec_list is a list of agg specs / programs; usually built via build('port ...'). The following tables show all m methods, and fields or properties (used interchangeably). Comments elucidate the meaning of more obscure entries.

1. Specification & creation

name (identity), label (display), agg_list (the units), n_units, unit_names, unit_names_ex (with total), unit_name_pipe, <unit> (each unit by name, e.g. A), value_type, exp_premium, program, pprogram, spec, spec_ex, m as_severity (collapse the total to a Severity), m create_from_sample, m create_frequency

2. Update

log2, bs, sev_calc, discretization_calc, normalize, padding, last_update, hash_rep_at_last_update, m update, m best_window (choose the shared combine grid), m snap, m remove_fuzz, m trim_density_df, m swap_density_df

3. Moments

actual = analytic, est = estimated from the FFT grid

actual_m, actual_cv, actual_sd, actual_var, actual_skew, est_m, est_cv, est_sd, est_var, est_skew, ex (E[X])

4. Statistical functions

m cdf, m sf, m pdf, m pmf, m q (VaR), m tvar, m tvar_threshold, m var_dict[1], prob_eq_0 (Pr(X=0)), m sample, sample_df, m sample_compare, m sample_density_compare

5. Validation

valid, validation_eps, validation_df, validation_explanation, bs_window_df, bs_description, bs_explanation, tail_class (worst of the units), tail_behavior_df, tail_description, tail_explanation, bounded

6. Output dataframes

density_df[1], density, summary_df (per unit + total, at a glance), stats_df, info, tail_df, m tail_periods_df, independent_density_df, independent_stats_df (the no-diversification comparison), m unit_density, m unit_density_df, m aligned_unit_density_df, m augmented_df, augmented_dfs (per-distortion cache), figure

7. Reinsurance

reins_kinds, reins_description (look-through, unit by unit), reins_summary_df, reins_density_df, reins_stats_df *(per-unit reinsurance is set on each Aggregate)*

8. Visualization

m plot, m scatter, figure

9. Risk and pricing

m price, m price_ccoc, m price_stand_alone, m price_pentagon, m price_pentagon_ex, m pricing_at, m pentagon_at, m prob_loss_assets, m apply_distortion, distortion, distortions, distortion_df, calibration_df, m calibrate_distortions, m analyze_distortion, m analyze_distortions, allocation_method (linear / lifted), m allocation_diagnostics, m allocation_bounds, m pricing_bounds[2], m bodoff, m add_exa, m add_exa_sample, m make_comonotonic_allocations

10. Approximations

m approximate (fit a single Aggregate to the total)

11. Meta

m help, m json, m save, m format_program, pprogram_html, note, tags, doc (the DecL trailer), label_map, labels, renamer, use_labels

Notes:

[1]: matches Aggregate.

[2]: both return objects from aggregate.bounds; see the Bounds card. allocation_bounds ranges the natural allocation by unit; pricing_bounds ranges the price of any other risk.

Abbreviations: m=mean, cv=coefficient of variation, sd=standard deviation, var=variance, skew(ness); VaR=value-at-risk; actual=analytic, est=FFT estimate; exa = E[X_i | X] (conditional expected allocation); CoC=cost of capital.

aggregate v.1.0.0a171
ease | confidence | precision
© Stephen J. Mildenhall
2026-07-29 11:33

PnL Class Cheat Sheet

m `PnL(*, name, source, groups=None, role=None, consideration=None, obligation=None, scale=None, result_name='result', label=None, label_map=None)`

Two construction forms: the ledger `groups=[Group(...), ...]`, and the sugar `role=/consideration=/obligation=` for exactly one `Group`. A m `Group(label, role, consideration, obligation)` holds m `Leg(label, func, bs, is2d)` cash flows.

A `PnL` is a position: one stochastic `source` plus an ordered ledger of signed `Groups`. Each group books a `consideration` and an `obligation` under a `role` (`sell` → +consideration, −obligation; `buy` → the contra), so the `EX` column adds straight down the sheet. All derived rows are per-atom partial sums, never arithmetic on marginals, so covariance is carried for free. Build with `build('pnl ...')` / `build('xpnl ...')`, or m `Aggregate.make_pnl`. `pnl` is the *consolidated* single-group net view, a flat `Consideration` / `Obligation` / `Margin` card; `xpnl` is the exploded sibling, same syntax, a multi-group walk with one group per step, so `summary_df` carries one `(Step, View)` block per step plus a closing `'All'`, and `stats_df` is three-level `(Step, View, Line)`. The following tables show all m `methods`, and fields or properties (used interchangeably). Comments elucidate the meaning of more obscure entries.

1. Specification & creation

`name` (identity), `label` (display), `source` (the stochastic generator: the *simplest sufficient* object, so a `GridDistribution`, an `Aggregate`, a bare `(values, probs)` pair, or a `BivariateDistribution` joint), `engine` (the full `Aggregate` / `Portfolio` behind it, for drill-down), `groups` (the declared `Group` specs, in ledger order), `role` (the single group's role; `None` on a multi-group ledger), `result_name`, `economics` (the resolved cession dict from `deposit` / `rol` / `rate` / `cede`), `program`, `pprogram`, `construction_description`, `construction_explanation` (how this P&L was built)

Two P&Ls over the same source concatenate with + (their ledgers chain).

2. Update

None. A `PnL` is a value object: it is assembled once from an already-updated source and holds signed per-atom rows plus their exact moments, with no retained engine dependency. Re-declare (or rebuild the engine) to change it.

`scale` *is the one committed setting: the unitizer for* `summary_df` *and* `scaled_stats_df` *(default: the expected signed grand total consideration).*

3. Moments

`est_m` (E[result], the signed grand net), `est_sd`, `est_cv` (meaningless near break-even, where the mean passes through zero; reported for symmetry), `est_skew`, `E_consideration` (E[total consideration], signed across groups)

Per-row moments (every leg, group total, group result, running net, and the grand rows) are the `EX` / `SD` / `CV` / `Skew` *columns of* `stats_df`.

4. Statistical functions

m `cdf` (Pr(result ≤ *x*)), m `sf`, m `q` (quantile = VaR of the result), m `tvar`, `prob_eq_0` (Pr(exact break-even)), `result`, `gd` (the grand result as a `GridDistribution`; `gd` is an alias)

All of these read the grand result*, the signed bottom line of the ledger. Adverse outcomes sit in the same tail for either role.*

5. Validation

`validation_df` (Est-vs-EX audit for every `bs > 0` leg)

EX is the exact signed mean straight off the atoms; `Est` *the mean of the rebucketed grid. The linear scheme matches means, so this reads very close, but it is visible. Legs with* `bs = 0` *are exact and absent. No reaching through to the source's own validation.*

6. Output dataframes

`summary_df` (the headline card: fixed rows that never vary with the ledger), `stats_df` (the full ledger × metrics sheet, in currency: the alignment / footing exhibit), `scaled_stats_df` (the same, unitized by `scale`), `density_df` (an ordered `{row label: GridDistribution}` — *not* one stapled frame; each row keeps its own exact grid), `info`, `figure`

7. Reinsurance

`economics` (the resolved `pc_occ` / `pc_agg` / `c_occ` / `c_agg` / `gross` / `ceded` dict)

A cession is an ordinary group booked `buy`*: consideration = ceded premium, obligation = recovery + commission. Reinstatements and variable rating (swing / slide / pc / corridor) are declared on the layer and consolidate into the same ledger; the resolved contract objects live on* `engine.reinstatement_terms` */* `engine.variable_terms`*.*

8. Visualization

m `plot` (grand result density and CDF; iterates `density_df`), `figure`

9. Risk and pricing

m `evaluate` (the Cherny–Madan breakeven acceptability panel)

A P&L is evaluated, not priced. *You price the* obligation *and ask which distortion drives the risk-adjusted net to zero: the breakeven stress the position survives. One row per family (*`ph`*,* `wang`*,* `dual`*,* `tvar`*), with* `param`*,* `error`*,* `gini_p`*,* `area`*. The breakeven* `gini_p` *is the single family-agnostic acceptability index.*

Needs one obligation leg over a regular lattice source.

10. Approximations

None. The ledger is exact on the source atoms; the only discretization is each leg's own rebucketing onto its grid, audited by `validation_df`.

11. Meta

m `help`, m `format_program`, `pprogram_html`, `note`, `tags`, `doc`, `hints` (the DecL trailer), `label_map`, `labels`, `renamer`, `use_labels`

`aggregate v.1.0.0a171`
ease | confidence | precision
© Stephen J. Mildenhall
2026-07-29 11:35

aggregate

`Distortion` Class Cheat Sheet

m `Distortion(name, shape=None, *, label=None, **natural)`

`Distortion` is a factory base class: the call `Distortion('ph', 0.9)` dispatches on `name` to a registered subclass (`PHDistortion`, `TVaRDistortion`, …) and returns an instance. A distortion is an increasing, concave $g : [0,1] \to [0,1]$ with $g(0) = 0$, $g(1) = 1$, used to define a coherent risk measure / pricing functional. Create one by family with a named constructor (s `Distortion.ph(0.9)`), or via DecL `distortion <NAME> <KIND> <PARAMS>`. The following tables show all m `methods`, s `static / class methods`, and fields or properties (used interchangeably). Comments elucidate the meaning of more obscure entries.

1. Specification & creation

`name` *the identity handle: the kind, or a given name*
`kind` *registry key for the subclass*
`shape` *the (single) shape parameter*
`label` *resolved display label*

Named constructors, by family (each s static): s `ph` (proportional hazard), s `wang` (Wang / normal shift), s `dual` (dual moment), s `power` (α on $[x_0, x_1]$), s `beta`, s `tvar` (TVaR at level p), s `bitvar` ($p_0 < p_1$, weight w on p_1), s `ccoc` (constant cost of capital from discount d), s `mean` (= TVaR at p=0), s `max` (= TVaR at p=1). *Combine / generate:*

s `mixture` (weighted mixture), s `minimum` (pointwise min), s `random_distortion`, s `random_distortion_ex` (random wtdTVaR), s `distortions_from_params` (dict from a calibration table), s `available_distortions` (list registered kinds), s `decl_spec` (spec dict from a DecL number list), s `from_spec` (construct from that spec)

2. Update

None — a `Distortion` is a closed-form function, not a discretized object. Everything is computed on demand.

3. Moments

None — the moments of g itself (D_g statistics, and the closed forms where they exist) are tabulated in `stats_df` / `summary_df`; see Output.

4. Statistical functions

m `g` (the distortion $g(x)$), m `g_inv` (inverse g^{-1}), m `g_dual` (dual $1 - g(1-x)$), m `g_prime` (derivative g', central-difference default), m `effective_g` (resolve the ask / bid view and the loss / payoff role to the effective g), m `make_q` (vector of risk-adjusted probabilities q for pricing), s `tvar_terms` (TVaR knot function $\min(s/(1-p), 1)$)

5. Validation

`pricing_ok` (the distortion yields well-defined prices), `strict_pricing` (enforce strict concavity / validity), `error` (numerical g vs. closed-form residual; see `summary_df`)

6. Output dataframes

`info` (text summary)[1], `summary_df`[1] (paired D_g, $D_{g^{-1}}$ columns plus checks), `stats_df`[1] (D_g statistics with closed form and error where analytics exist), `density_df`[1] (x, g, g^{-1}, g^*, g', kusuoka), `tvar_info_df` (affine pieces of a weighted-TVaR: knots, slopes, intercepts), `decl_params`, `param_name`

7. Reinsurance

None

8. Visualization

m `plot` (g, dual, and diagonal)[1], m `plot_affine` (upper affine envelope of a wtdTVaR)

9. Risk and pricing

m `price` (distorted ask / bid prices of a distribution), m `calibrate` (solve `shape` so the distorted price hits a `premium_target`), s `calibrate_set` (calibrate a whole family to one target), `premium_target`, `assets`, `a` (capital level), `mass`, `has_mass`, `has_mass_default` (point mass $g(0^+) > 0$), `gini_p` (Gini coefficient)

10. Approximations

None

11. Meta

m `id` (short structural ID string), `med_name`, `long_name` (display labels), `documented` (listed by `available_distortions`), m `help`, m `format_program`, `program`, `pprogram`, `pprogram_html`, `note`, `tags`, `doc`, `hints` (the DecL trailer)[2], `label_map`, `labels`, `renamer`, `use_labels`, m `min_index`, m `quick_gS`, m `quick_ra` (numba-backed numerical helpers)

Notes:

[1]: `info`, `summary_df`, `stats_df`, `density_df`, and `plot` mirror the common reporting interface shared with `Aggregate` and `Portfolio`: the same names do the analogous job on each class.

[2]: a `dist` line carries the same trailer as every other DecL statement, so a distortion can be described, tagged, and documented like an aggregate.

$g(0) = 0$, $g(1) = 1$, increasing and concave; dual $g^*(x) = 1 - g(1-x)$; price $= \int_0^\infty g(S(x))\,dx$.

`aggregate v.1.0.0a171`
ease | confidence | precision
© Stephen J. Mildenhall
2026-07-29 11:34

*Bounds Classes Cheat Sheet

pricing and allocation bounds

Three classes in `aggregate.bounds`, two papers, one question: *given that a risk is priced to P, what else is pinned down?* All three parameterise the extreme consistent distortions as biTVaRs $(1-w_1)\,\mathrm{TVaR}_{p_0} + w_1\,\mathrm{TVaR}_{p_1}$ with $p_0 \le p_* \le p_1$.

Not DecL-creatable — but DecL is at both ends. Every *input* is a DecL object (`build('port ...')`, `build('agg ...')`), and every *output* is a biTVaR distortion, which is DecL-writable: `ab.distortion(P, 'A', 'upper')` returns a `Distortion` you can spell `dist D bitvar <p0> <p1> <w1>` and hand to `port.price`. That round trip is the creation story; the constructors below are the only entry points.

How they differ. `Bounds` studies the distortion family itself (the envelopes of g), on a p-knot grid with a root find: approximate. The two hull classes study the *image* of that family, on exact CDF-breakpoint vertices with monotone convex hulls: exact, and P enters only as a cheap slice.

The geometry. Represent a distortion by its Kusuoka measure μ, so $\rho_\mu(X) = \int \mathrm{TVaR}_p(X)\,\mu(dp)$. Pricing X to P is *one affine constraint* on probability measures, so the extreme consistent measures are two-point mixtures, biTVaRs. Write $T(p) = \mathrm{TVaR}_p(X)$ and plot the score $y_j(p)$ against $T(p)$ as p sweeps $[0,1]$: the vertical slice at $T = P$ through the convex hull of that curve is $[\min, \max]$, and the hull edge crossing $T = P$ names the achieving biTVaR. On the FFT grid both coordinates are affine in $u = 1/(1-p)$ within an atom, so the curve is exactly piecewise linear and the hull of the curve equals the hull of its breakpoint vertices: no p-grid, no interpolation error. Here p_* solves $T(p_*) = P$, exeqa is $\mathsf{E}[X_i \mid X = x]$, and NA is the natural allocation. **The Gini lens** is the case $X = U[0,1]$, where $T(p) = (1+p)/2$ is affine in p, the constraint collapses to the mean Kusuoka level $\mathsf{E}_\mu[p] = 2P - 1$, and the bounds read off the envelope gap of Y's own TVaR curve: an X-free reading of why distortions pricing Y disagree.

The columns show all (m) `methods`, and fields or properties (used interchangeably), under the eleven standard category headings. Categories **2** (update), **3** (moments), **7** (reinsurance), and **10** (approximations) are empty for all three and are omitted: a bounds object is built once from an already-updated risk and is then pure geometry, with nothing to re-discretize, no moments of its own, no cessions, no fitted approximation.

`Bounds` — the distortion family (IME 2022)

(m) `Bounds(obj, premium, *, a=inf, unit='total', n_p=256, n_s=513)`

1. Specification & creation. `obj` is a `Portfolio`, `Aggregate`, or pmf `Series` / `DataFrame`; the premium is *baked in at construction* ($\mathsf{E}[X] < \texttt{premium} \le a$). `name`, `unit`, `premium`, `a` (asset cap), `Fb` ($F(a)$), `n_p`, `n_s` (grid sizes).

4. Statistical functions. `p_star` (the p with $\mathrm{TVaR}_p(\min(X,a)) = \text{premium}$; a cached property, found by root search), `p_knots` (the p axis), `s_grid` (the s axis), `tvar_x_p`, `tvar_hinges` ($\min(1, s/(1-p))$), (m) `distortion(pl, pu)` (the biTVaR with those knots and the matching weight).

5. Validation. *None:* the p-knot grid plus `brentq` is approximate by construction. Refine with `n_p` / `n_s`.

6. Output dataframes. `info`, `tvar_df` (p vs. TVaR), `weight_df` (one row per bracketing (p_0, p_1) pair), `cloud_df` (shape (n_s, K), one column per bracket).

8. Visualization. (m) `plot_envelope` (the three-panel cloud figure from the paper), (m) `plot_weights`.

9. Risk and pricing. `min_envelope` (pointwise minimum of the cloud; min of concaves is concave, so this *is* a `Distortion`), `max_envelope` (pointwise maximum, an interpolating *callable only*: max of concaves need not be concave), `min_envelope_hinges`.

11. Meta. (m) `help`.

`AllocationBounds` — by unit

(m) `AllocationBounds(port, *, a=inf, units=None, s_floor=1e-14)`
(m) `Portfolio.allocation_bounds(*, a=0, p=0, units, s_floor)`

1. Specification & creation. Portfolio only (it reads the `exeqa_*` columns, so the portfolio must be updated). Given the total is priced to P, what is the range of *natural allocation* premium to each unit? `port`, `units`, `a`, `s_floor`. *The premium is a call-time argument, not baked in.*

4. Statistical functions. (m) `p_star(P)` (exact per-atom inversion of $T(p) = P$), (m) `bounds(P)` (the answer: `lower` / `upper` / `width` by unit), (m) `bitvars(P)` (the achieving (p_0, p_1, w_1) per unit and side), (m) `na_grid(p_grid)`.

5. Validation. (m) `check(P)` (audit: reprice with the achieving biTVaRs), `additivity_error` ($\max_p |\sum_i a_i(p) - T(p)|$, inherited `density_df` FFT noise; $\sim 10^{-6}$ relative).

6. Output dataframes. `info`, `curve_df` (vertex table indexed by p: `exeqa_total` $= T(p)$, one `exeqa_<unit>` column each), `premium_range` ($= [\mathsf{E}[X], T_{\max}]$).

8. Visualization. (m) `plot(items, P)` (curves, envelopes, the P-slice). **9. Risk and pricing.** (m) `distortion(P, item, bound)`. **11. Meta.** (m) `help`.

`PricingBounds` — any second risk

(m) `PricingBounds(x_source, y_sources, *, a=inf, s_floor=1e-14, n_grid=1024)`
(m) `Portfolio.pricing_bounds(y_sources, *, a=0, p=0, s_floor, n_grid)`

1. Specification & creation. The same question for *any* second risk: given X is priced to P, what is the range of the price of Y? A source is an `Aggregate`, `Portfolio`, pmf `Series`, or `'uniform'`; `y_sources` takes one, a list, or a naming dict. `a`, `s_floor`, `n_grid`.

4. Statistical functions. (m) `p_star(P)`, (m) `bounds(P)`, (m) `bitvars(P)` *(as for* `AllocationBounds`*)*, (m) `mean_kusuoka_level(P)` $(= 2P - 1)$, `gini_level`.

5. Validation. (m) `check(P)`. *Bounded (a finite) reprices to $\sim 10^{-12}$. Unbounded with a heavy-tailed Y the upper bound is genuinely tail-driven and grid-sensitive: cap the risks or raise `s_floor`.*

6. Output dataframes. `info`, `curve_df` (vertex table: `tvar_<X>` plus one `price_<Y>` column per Y), `premium_range`.

8. Visualization. (m) `plot(items, P)`. **9. Risk and pricing.** (m) `distortion(P, item, bound)`. **11. Meta.** (m) `help`.

`aggregate v.1.0.0a171`
ease | confidence | precision
© Stephen J. Mildenhall
2026-07-29 11:34